Hadoop & Spark

A Complete Guide for Computer Science Students

Installation • Architecture • Programming • Troubleshooting

Submitted To: **Mahek Kausar**

Submitted By:

Name	Reg. No.
Abdullah Shafiq	L1F22BSCS0583
Waleed Ahmad	L1F22BSCS0387
Arslan Asif	L1F22BSCS0723
Malik Faisal Mukhtar	L1F22BSCS1102

University of Central Punjab

Faculty of Information Technology

Lahore, Pakistan

Parallel & Distributed Computing — Spring 2026

Hadoop 3.3.6 • Spark 3.5.x • 2026

Contents

Table of Contents 1

1 Apache Hadoop 2
- 1.1 What is Hadoop? 2
  - 1.1.1 Why Hadoop? 2
- 1.2 Core Components 2
  - 1.2.1 HDFS - Hadoop Distributed File System 2
  - 1.2.2 YARN - Yet Another Resource Negotiator 3
  - 1.2.3 MapReduce 3
- 1.3 Hadoop Architecture 3
- 1.4 MapReduce Data Flow 4
- 1.5 HDFS Read & Write Internals 4
  - 1.5.1 Write Pipeline 4
  - 1.5.2 Read Path 4
- 1.6 The Hadoop Ecosystem 5

2 Hadoop Installation 6
- 2.1 System Requirements 6
- 2.2 Windows Installation via WSL2 6
  - 2.2.1 Enable WSL2 and Install Ubuntu 6
  - 2.2.2 First-Time Ubuntu Setup 7
- 2.3 Linux Preparation (Native) 7
- 2.4 Step-by-Step Hadoop Setup 7
  - 2.4.1 Step 1 - Install Java 7
  - 2.4.2 Step 2 - Configure SSH (passwordless) 8
  - 2.4.3 Step 3 - Download and Extract Hadoop 9
  - 2.4.4 Step 4 - Set Environment Variables 9
  - 2.4.5 Step 5 - Configure Hadoop XML Files 10
  - 2.4.6 Step 6 - Create Data Directories and Format NameNode 12
  - 2.4.7 Step 7 - Start and Verify Hadoop 12
- 2.5 Hands-On Lab: WordCount 14
- 2.6 Common HDFS Commands Reference 15

3 Apache Spark 16
- 3.1 What is Apache Spark? 16
- 3.2 Spark vs. Hadoop MapReduce 16
- 3.3 Spark Architecture 16
  - 3.3.1 Cluster Architecture Deep Dive 17
  - 3.3.2 Lazy Evaluation and DAG Scheduling 17

4 Spark Installation 18
- 4.1 Prerequisites 18
- 4.2 Windows Installation (WSL2) 18

4.3 Linux Installation (Fedora / Ubuntu Native) . . . 18
4.3.1 Method A - Install via pip (Easiest for students) . . . 18
4.3.2 Method B - Manual Spark Installation (Recommended for full features) . . . 18
4.4 Verify Spark is Working . . . 19
4.4.1 PySpark Interactive Shell . . . 19
4.4.2 Submit a Standalone Script . . . 20
4.5 Common Spark Installation Errors . . . 21

5 PySpark Programming Guide 22
5.1 RDDs - Resilient Distributed Datasets . . . 22
5.1.1 What is an RDD? . . . 22
5.1.2 Creating RDDs . . . 22
5.1.3 Key Transformations and Actions . . . 23
5.2 DataFrames and Spark SQL . . . 23
5.2.1 Creating DataFrames . . . 24
5.2.2 DataFrame Operations . . . 24
5.2.3 Spark SQL . . . 25
5.3 Data Sources and File I/O . . . 25
5.4 Performance Optimization . . . 26
5.4.1 Partitioning Strategy . . . 26
5.4.2 Caching . . . 26
5.4.3 Avoid Common Anti-Patterns . . . 27
5.5 Performance Optimization . . . 27
5.5.1 Partitioning Strategy . . . 27
5.5.2 Understanding Shuffles . . . 27
5.5.3 Caching . . . 28
5.5.4 Catalyst Optimizer and Tungsten . . . 28
5.5.5 Avoid Common Anti-Patterns . . . 29
5.6 Structured Streaming . . . 29
5.6.1 Batch vs. Streaming . . . 29
5.6.2 Streaming Pipeline Example . . . 29
5.6.3 Output Modes and Sinks . . . 30
5.7 MLlib - Machine Learning with Spark . . . 30

6 Spark on Hadoop YARN 32
6.1 How Spark Integrates with YARN . . . 32
6.2 Submitting Spark Jobs to YARN . . . 32
6.3 Tuning Spark Resource Allocation . . . 33
6.4 Spark Web UI on YARN . . . 35

7 Practice Projects 37
7.1 Project 1 - ETL Pipeline with Spark . . . 37
7.2 Project 2 - WordCount: RDD vs. DataFrame . . . 38
7.3 Project 3 - Analysing a Real Dataset . . . 38
7.4 Managed Spark and Cloud Platforms . . . 40
7.5 Recommended Learning Path . . . 41

Appendix A: Quick Cheat Sheet 42

References 44

Chapter 1

Apache Hadoop

1.1 What is Hadoop?

Imagine you have 1 TB of raw data and a single laptop - within minutes the machine crawls, and if the disk fails, everything is gone. **Apache Hadoop** solves both problems. It is an open-source framework from the Apache Software Foundation that *stores* huge datasets across many commodity machines (via HDFS) and *processes* them in parallel (via YARN + MapReduce) - so the workload is shared, and data is replicated for safety.

Key fact

A single Hadoop cluster at large companies may span **thousands of nodes**, collectively storing petabytes of data at a fraction of the cost of enterprise storage systems.

1.1.1 Why Hadoop?

Aspect	Without Hadoop	With Hadoop
Scale	One machine, limited RAM/disk	Hundreds–thousands of nodes
Fault Tolerance	Single point of failure	Replication (default: 3 copies)
Speed	Sequential processing	Parallel map + reduce
Cost	Expensive SAN/NAS	Commodity x86 hardware

1.2 Core Components

Hadoop has three fundamental pillars:



Figure 1.1. *The three core pillars of Apache Hadoop*

1.2.1 HDFS - Hadoop Distributed File System

HDFS is inspired by Google's GFS paper. It splits files into large **blocks** (default 128 MB) and distributes them across **DataNodes**, keeping multiple copies (replication factor = 3 by default). A single **NameNode** holds all filesystem metadata.

1.2.2 YARN - Yet Another Resource Negotiator

YARN decouples resource management from processing logic. The **ResourceManager** is the cluster master; **NodeManagers** on each worker node launch and supervise *containers* - units of allocated CPU + memory.

1.2.3 MapReduce

MapReduce is a *programming model*, not a database. Jobs are split into:

- **Map** - process each input record independently, emit key-value pairs
- **Shuffle & Sort** - group identical keys across all mappers
- **Reduce** - aggregate values per key into the final output

1.3 Hadoop Architecture

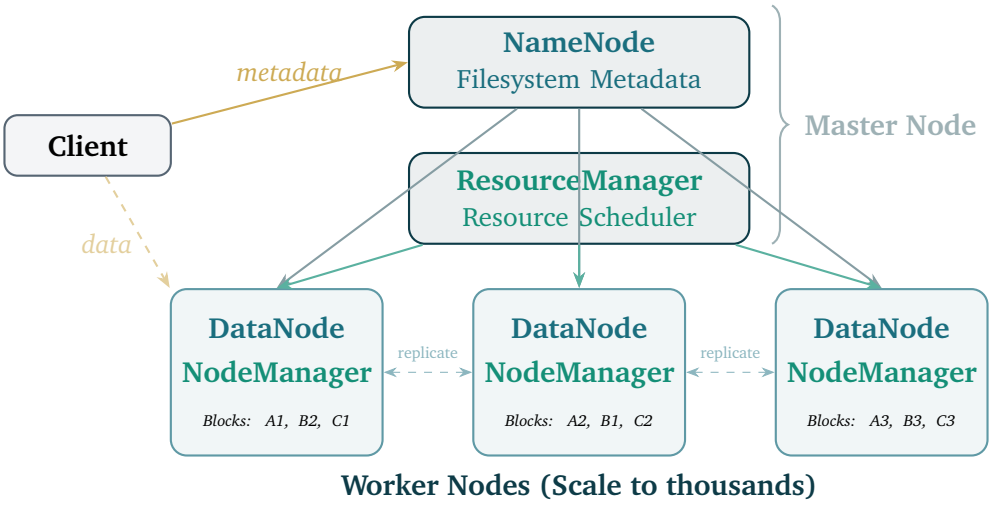


Figure 1.2. *Hadoop master-worker architecture with HDFS and YARN daemons*

Daemon	Layer	Responsibility
NameNode	HDFS Master	Stores metadata: block locations, file names, permissions
DataNode	HDFS Worker	Stores actual data blocks on disk; sends heartbeats
SecondaryNameNode	HDFS Helper	Periodic checkpoints (not a failover backup)
ResourceManager	YARN Master	Global resource scheduler across the cluster
NodeManager	YARN Worker	Launches containers; reports resource usage

Table 1.1. *Hadoop daemons and their roles*

1.4 MapReduce Data Flow



Figure 1.3. *MapReduce data flow illustrated with a WordCount example*

1.5 HDFS Read & Write Internals

Understanding how data actually moves in HDFS helps you debug real-world problems.

1.5.1 Write Pipeline

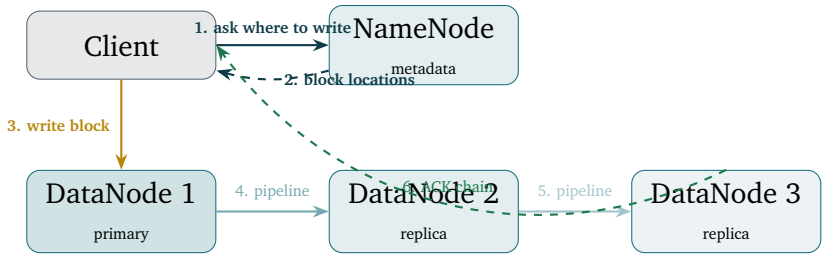


Figure 1.4. *HDFS write pipeline: blocks streamed and replicated in a chain*

1. Client contacts NameNode to create a new file entry.
2. NameNode returns the list of DataNodes to store each block.
3. Client streams data directly to DataNode 1 (never through NameNode).
4. DataNode 1 simultaneously forwards to DataNode 2, which forwards to DataNode 3.
5. Once all replicas acknowledge, a success ACK is sent back to the client.
6. Client notifies NameNode that the block is complete.

Why does data bypass the NameNode?

The NameNode only handles metadata (file names, block locations). Routing actual data through it would create a fatal bottleneck. This separation is one of HDFS's core design wins and lets the cluster scale to thousands of nodes.

1.5.2 Read Path

On read, the client fetches the block location list from the NameNode, then contacts the *closest* DataNode directly (same rack preferred). If that DataNode is slow or down, the client automatically fails over to a replica.

Spark and MapReduce exploit **data locality** - they schedule tasks on the same node that holds the blocks, minimising network I/O.

1.6 The Hadoop Ecosystem

Hadoop is the foundation of a rich family of tools you will encounter in industry.

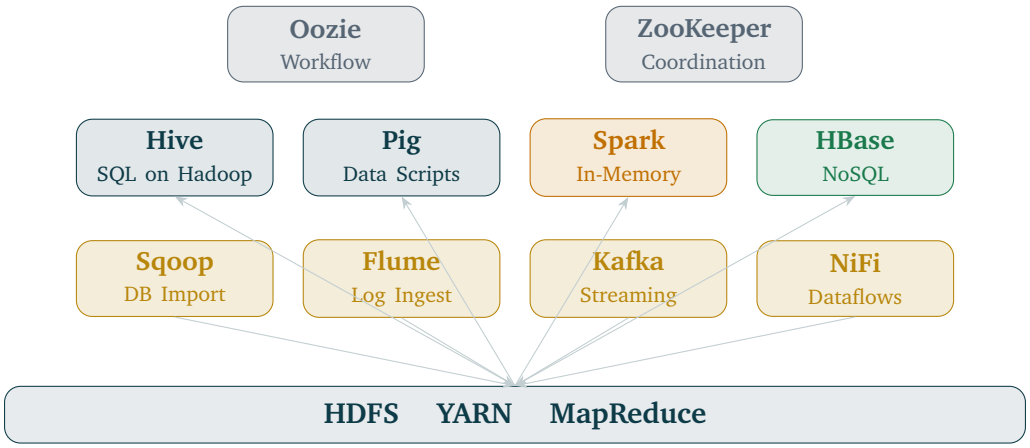


Figure 1.5. *The Hadoop ecosystem - tools layered above HDFS and YARN*

Tool	Category	Purpose
Hive	SQL Engine	SQL-like queries that compile to MapReduce/Tez jobs
HBase	NoSQL DB	Column-family store on HDFS; random reads/writes
Pig	Scripting	High-level dataflow language (Pig Latin) for ETL
Sqoop	Ingestion	Bulk import/export between RDBMS and HDFS
Flume	Ingestion	Reliable collection of log and event streams
Kafka	Streaming	High-throughput distributed event/message bus
Oozie	Workflow	Schedule Hadoop job workflows (DAGs of jobs)
ZooKeeper	Coordination	Leader election, distributed locking, config sync

Table 1.2. *Hadoop ecosystem tools*

Chapter 2

Hadoop Installation

What we install

We install Hadoop 3.3.6 in **pseudo-distributed mode**: all daemons run on one machine, but HDFS and YARN are fully functional - ideal for learning and development.

2.1 System Requirements

Item	Minimum	Recommended
OS	Ubuntu 22.04 (WSL) / Fedora 40+	Ubuntu 22.04 LTS / Fedora 41
RAM	4 GB	8 GB
Disk	15 GB free	25 GB free
Java	JDK 17	OpenJDK 17
Network	Required (initial setup)	Broadband

2.2 Windows Installation via WSL2

If you are on Windows, the easiest way to follow this guide is to use **Ubuntu** via WSL2, which is officially supported by Microsoft.

2.2.1 Enable WSL2 and Install Ubuntu

Open **PowerShell as Administrator** and run:

Listing 2.1. *Enable WSL2 and install Ubuntu*

```
# Enable WSL and Virtual Machine Platform features
dism.exe /online /enable-feature
   /featurename:Microsoft-Windows-Subsystem-Linux /all
   /norestart
dism.exe /online /enable-feature
   /featurename:VirtualMachinePlatform /all /norestart

# Restart your PC, then set WSL2 as default and install Ubuntu
wsl --set-default-version 2
wsl --install -d Ubuntu-22.04
```

Restart your PC after enabling the features. If WSL2 is not available, download the *WSL2 Linux kernel update package* from the Microsoft website and install it first.

2.2.2 First-Time Ubuntu Setup

After the Ubuntu terminal opens for the first time:

1. Create a Unix username (e.g., `hadoopuser`) - **no spaces, lowercase**
2. Set a strong password (invisible while typing - this is normal)
3. Update the package index:

Listing 2.2. *Update Ubuntu in WSL2*

```
sudo apt update && sudo apt upgrade -y
```

2.3 Linux Preparation (Native)

If you are running native Linux, open a terminal and update your system:

Listing 2.3. *Update packages (Ubuntu vs Fedora)*

```
# \lin On Ubuntu:
sudo apt update && sudo apt upgrade -y

# \lin On Fedora:
sudo dnf update -y
```

Ubuntu vs Fedora commands

Throughout this guide, commands will be provided for both **Ubuntu** (`apt`) and **Fedora** (`dnf`) where they differ.

2.4 Step-by-Step Hadoop Setup

2.4.1 Step 1 - Install Java

Listing 2.4. *Install OpenJDK 17 (Ubuntu vs Fedora)*

```
# \lin On Ubuntu:
sudo apt install openjdk-17-jdk -y

# \lin On Fedora:
sudo dnf install java-17-openjdk-devel -y

java -version    # Expected: openjdk version "17.x.x"
```

Error: `java: command not found`

Cause: Installation failed or PATH not updated.
Fix (Ubuntu): Run `sudo apt install -reinstall openjdk-17-jdk -y` **Fix (Fedora):** Run `sudo dnf reinstall java-17-openjdk-devel -y` Then open a new terminal.



Figure 2.1. *Verifying Java 17 installation and `JAVA_HOME`*

2.4.2 Step 2 - Configure SSH (passwordless)

Hadoop's scripts use SSH to start/stop daemons even on a single machine.

Listing 2.5. *Install SSH and configure passwordless login*

```
# \lin On Ubuntu:
sudo apt install openssh-server -y
sudo service ssh start    # Start SSH service (WSL2 only)

# \lin On Fedora:
sudo dnf install openssh-server -y
sudo systemctl enable --now sshd    # Enable and start SSH
    service

ssh-keygen -t rsa -b 4096 -N "" -f ~/.ssh/id_rsa
cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys

# Test: should connect without password prompt
ssh localhost
exit
```

Error: SSH asks for password every time

Cause: Permissions on `authorized_keys` or `.ssh` are too open.
Fix:

```
chmod 700 ~/.ssh
chmod 600 ~/.ssh/authorized_keys
```

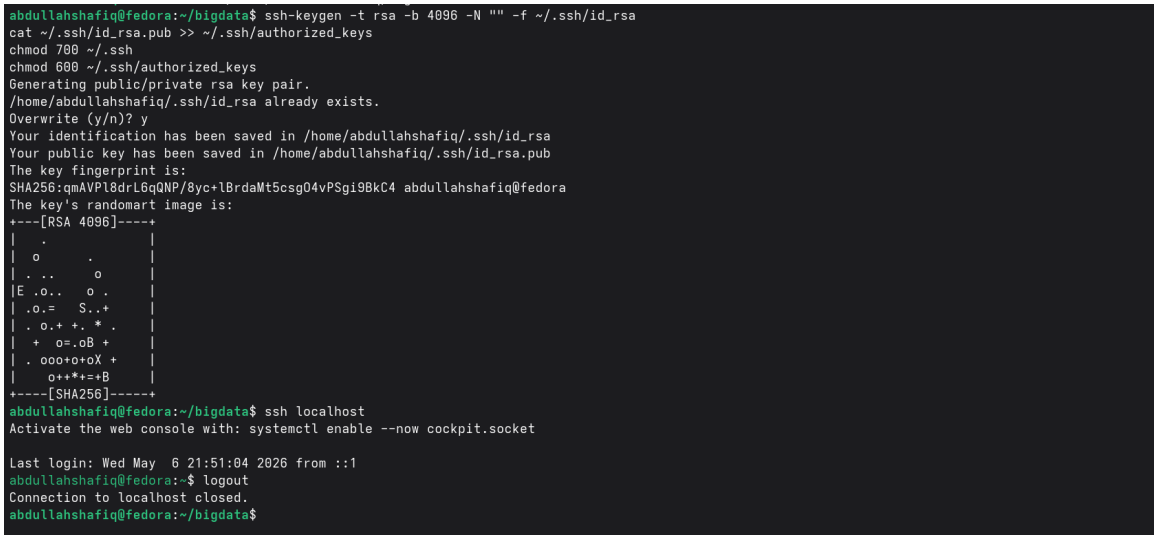


Figure 2.2. *SSH key generation and passwordless login to localhost*

2.4.3 Step 3 - Download and Extract Hadoop

Listing 2.6. *Download Hadoop 3.3.6*

```
cd ~
wget
   https://downloads.apache.org/hadoop/common/hadoop-3.3.6/hadoop-3.3.6.tar.
tar -xzf hadoop-3.3.6.tar.gz
mv hadoop-3.3.6 hadoop
ls ~/hadoop     # Confirm folder exists
```

If the Apache mirror is slow, try a closer mirror at https://hadoop.apache.org/releases.html (click **Download** link for the nearest mirror).

2.4.4 Step 4 - Set Environment Variables

Listing 2.7. *Append to ~/.bashrc*

```
cat >> ~/.bashrc << 'EOF'

# -- Hadoop Environment --
# Uncomment ONE of the following JAVA_HOME lines depending on
   your OS:
# export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64   # Ubuntu
# export JAVA_HOME=/usr/lib/jvm/java-17-openjdk         # Fedora

export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64     #
   Default to Ubuntu
export HADOOP_HOME=$HOME/hadoop
export HADOOP_CONF_DIR=$HADOOP_HOME/etc/hadoop
export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin
```

```
EOF

source ~/.bashrc
hadoop version    # Should print: Hadoop 3.3.6
```



Figure 2.3. *`hadoop version` confirming a successful 3.3.6 installation*

Finding your JAVA_HOME

Ubuntu: Use `update-java-alternatives -list`. Usually `/usr/lib/jvm/java-17-openjdk-amd64`.
Fedora: Use `alternatives -display java`. Usually `/usr/lib/jvm/java-17-openjdk`.

JDK 17+ requires `-add-opens` flags for Hadoop 3.3.6.
JDK 17 enforces the Java Module System strictly, which breaks some Hadoop internals. Add the following lines to `$HADOOP_HOME/etc/hadoop/hadoop-env.sh`:

```
# Fix for JDK 17+ module access restrictions
export HADOOP_OPTS="$HADOOP_OPTS --add-opens
   java.base/java.lang=ALL-UNNAMED"
export HADOOP_OPTS="$HADOOP_OPTS --add-opens
   java.base/java.util=ALL-UNNAMED"
export HADOOP_OPTS="$HADOOP_OPTS --add-opens
   java.base/java.io=ALL-UNNAMED"
export HADOOP_OPTS="$HADOOP_OPTS --add-opens
   java.base/java.net=ALL-UNNAMED"
export HADOOP_OPTS="$HADOOP_OPTS --add-opens
   java.base/sun.nio.ch=ALL-UNNAMED"
export HADOOP_OPTS="$HADOOP_OPTS --add-opens
   java.base/sun.security.ssl=ALL-UNNAMED"
```

Without these flags, you may see `InaccessibleObjectException` or `IllegalAccessError` when starting Hadoop daemons.

2.4.5 Step 5 - Configure Hadoop XML Files

All configuration lives in `$HADOOP_HOME/etc/hadoop/`.

core-site.xml - Default filesystem

Listing 2.8. *core-site.xml*

```
<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://localhost:9000</value>
  </property>
  <property>
    <name>hadoop.tmp.dir</name>
    <value>/home/${user.name}/hadoopdata/tmp</value>
  </property>
</configuration>
```

hdfs-site.xml - HDFS settings

Listing 2.9. *hdfs-site.xml (pseudo-distributed: replication=1)*

```
<configuration>
  <property>
    <name>dfs.replication</name>
    <value>1</value>
  </property>
  <property>
    <name>dfs.namenode.name.dir</name>
    <value>file:///home/${user.name}/hadoopdata/namenode</value>
  </property>
  <property>
    <name>dfs.datanode.data.dir</name>
    <value>file:///home/${user.name}/hadoopdata/datanode</value>
  </property>
  <property>
    <name>dfs.namenode.http-address</name>
    <value>localhost:9870</value>
  </property>
</configuration>
```

mapred-site.xml - MapReduce framework

Listing 2.10. *mapred-site.xml*

```
<configuration>
  <property>
    <name>mapreduce.framework.name</name>
    <value>yarn</value>
  </property>
  <property>
    <name>mapreduce.application.classpath</name>
    <value>$HADOOP_HOME/share/hadoop/mapreduce/*:
           $HADOOP_HOME/share/hadoop/mapreduce/lib/*</value>
  </property>
</configuration>
```

yarn-site.xml - YARN settings

Listing 2.11. *yarn-site.xml*

```
<configuration>
  <property>
    <name>yarn.nodemanager.aux-services</name>
    <value>mapreduce_shuffle</value>
  </property>
  <property>
    <name>yarn.resourcemanager.hostname</name>
    <value>localhost</value>
  </property>
  <property>
    <name>yarn.nodemanager.env-whitelist</name>
    <value>JAVA_HOME,HADOOP_COMMON_HOME,HADOOP_HDFS_HOME,
HADOOP_CONF_DIR,HADOOP_YARN_HOME,HADOOP_MAPRED_HOME</value>
  </property>
</configuration>
```

2.4.6 Step 6 - Create Data Directories and Format NameNode

Listing 2.12. *Format the NameNode (one-time only)*

```
mkdir -p ~/hadoopdata/{namenode,datanode,tmp}

# Format NameNode - do this ONLY ONCE
hdfs namenode -format
```

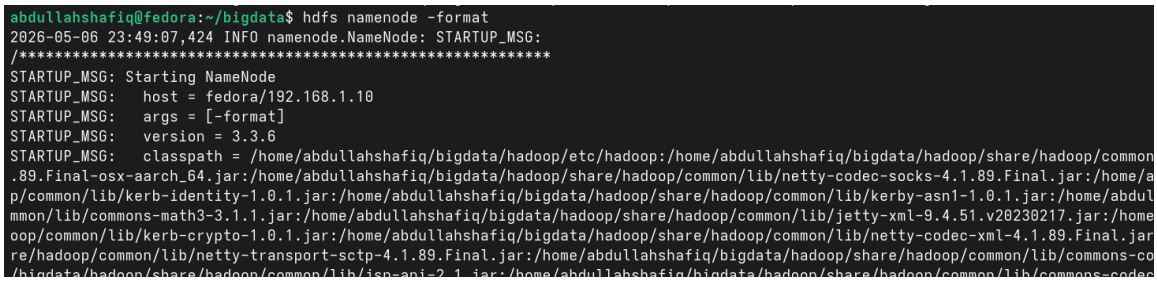


Figure 2.4. *NameNode formatting - look for "successfully formatted" confirmation*

Never format the NameNode again after you have data in HDFS - it wipes all filesystem metadata. If you need to reset, delete `~/hadoopdata/` first.

2.4.7 Step 7 - Start and Verify Hadoop

Listing 2.13. *Start all Hadoop daemons*

```
start-dfs.sh
start-yarn.sh

# Verify all 5 daemons are running
jps
# Expected output (PIDs will differ):
# 2341 NameNode
# 2456 DataNode
# 2567 SecondaryNameNode
# 2678 ResourceManager
# 2789 NodeManager
```

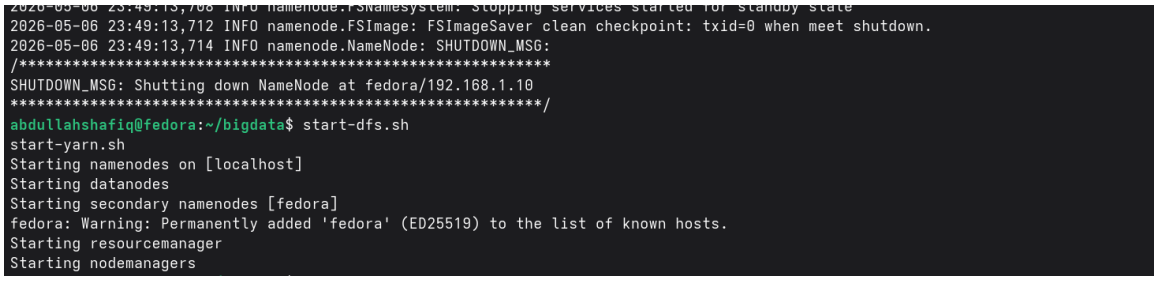


Figure 2.5. *Starting HDFS and YARN daemons with* `start-dfs.sh` *and* `start-yarn.sh`

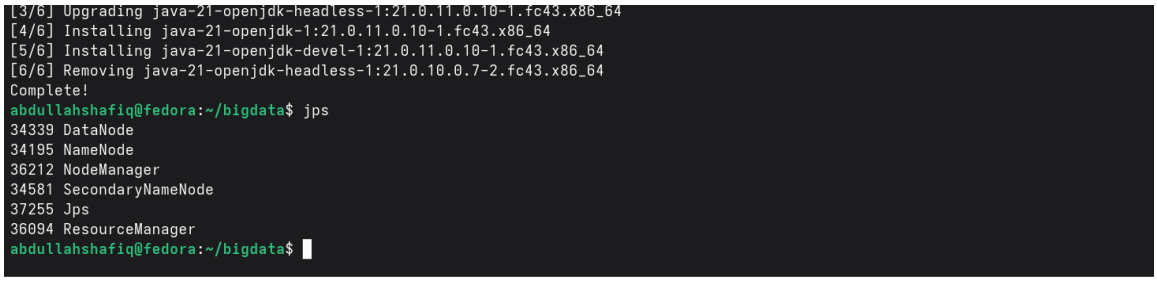


Figure 2.6. *`jps` output showing all five Hadoop daemons running successfully*

Web UI	URL
HDFS NameNode	http://localhost:9870
YARN ResourceManager	http://localhost:8088
MapReduce JobHistory	http://localhost:19888

Error: `DataNode` missing from `jps`

Cause: Incompatible cluster ID from a previous format.
Fix:

```
stop-dfs.sh
rm -rf ~/hadoopdata/datanode/*
hdfs namenode -format -force
start-dfs.sh
```

Error: `NameNode is in safe mode`

Cause: Startup safety check still running.
Fix: Wait 60 seconds, then run:

```
hdfs dfsadmin -safemode leave
```

WSL2: Web UI not reachable from browser

Cause: WSL2 network isolation.
Fix: Use `localhost` (Windows 10/11 automatically forwards WSL2 ports). If still failing, find WSL IP via `hostname -I` and use that IP instead.

If the Web UI is not reachable, ensure the Fedora firewall allows the ports:

```
sudo firewall-cmd --add-port=9870/tcp --add-port=8088/tcp
   --zone=public
```

2.5 Hands-On Lab: WordCount

Listing 2.14. *Run the classic Hadoop WordCount job*

```
# Create a sample input file
echo "Hadoop is great for big data processing
Hadoop HDFS stores data across many nodes
YARN manages cluster resources efficiently" > sample.txt

# Upload to HDFS
hdfs dfs -mkdir -p /user/$USER/input
hdfs dfs -put sample.txt /user/$USER/input/

# Run WordCount (built-in example JAR)
hadoop jar \
  $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-*.jar
     \
  wordcount \
  /user/$USER/input \
  /user/$USER/output_wc

# View results
hdfs dfs -cat /user/$USER/output_wc/part-r-00000
```



Figure 2.7. *Running WordCount MapReduce job and viewing results with* `hdfs dfs -cat`

Expected output:

```
HDFS       1
YARN       1
Hadoop     3
across     1
big        1
cluster    1
...
```

Listing 2.15. *Stop Hadoop when done*

```
stop-yarn.sh
stop-dfs.sh
```

2.6 Common HDFS Commands Reference

Command	Description
`hdfs dfs -ls /path`	List files/directories
`hdfs dfs -mkdir -p /path`	Create directory (recursive)
`hdfs dfs -put local.txt /hdfs/path`	Upload file to HDFS
`hdfs dfs -get /hdfs/path local.txt`	Download from HDFS
`hdfs dfs -cat /path/file`	Print file contents
`hdfs dfs -rm -r /path`	Delete file or directory
`hdfs dfs -df -h`	Show HDFS disk usage
`hdfs dfsadmin -report`	Cluster health report

Table 2.1. *Essential HDFS shell commands*

Chapter 3

Apache Spark

3.1 What is Apache Spark?

Apache Spark is an open-source **unified analytics engine** for large-scale data processing. Originally developed at UC Berkeley's AMPLab (2009) and donated to the Apache Software Foundation, Spark has become the de-facto standard for in-memory distributed computing - from batch ETL to real-time streaming, SQL analytics, and machine learning.

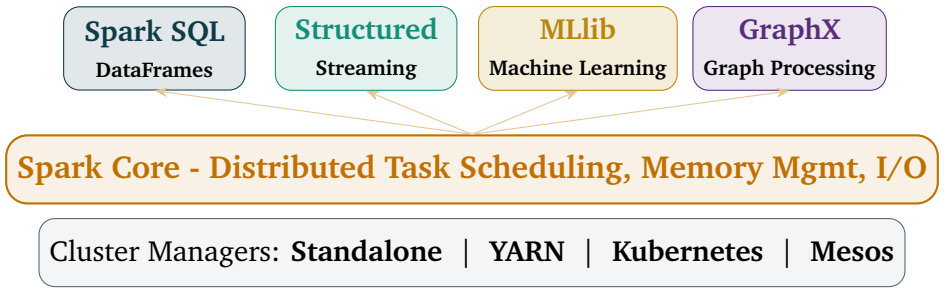


Figure 3.1. *Apache Spark unified platform: one engine for every workload*

3.2 Spark vs. Hadoop MapReduce

Feature	Apache Spark	Hadoop MapReduce
Processing model	In-memory	Disk-based
Speed	**10–100× faster**	Baseline
Ease of use	High (PySpark API)	Low (verbose Java)
Workloads	Batch, Stream, ML, SQL	Batch only
Fault tolerance	RDD Lineage	Disk replication
Interactive queries	Yes (Spark Shell)	No
Real-time streaming	Yes (Structured Streaming)	No
Language support	Python, Scala, Java, R	Java primarily

Table 3.1. *Side-by-side comparison of Spark and MapReduce*

3.3 Spark Architecture

3.3.1 Cluster Architecture Deep Dive

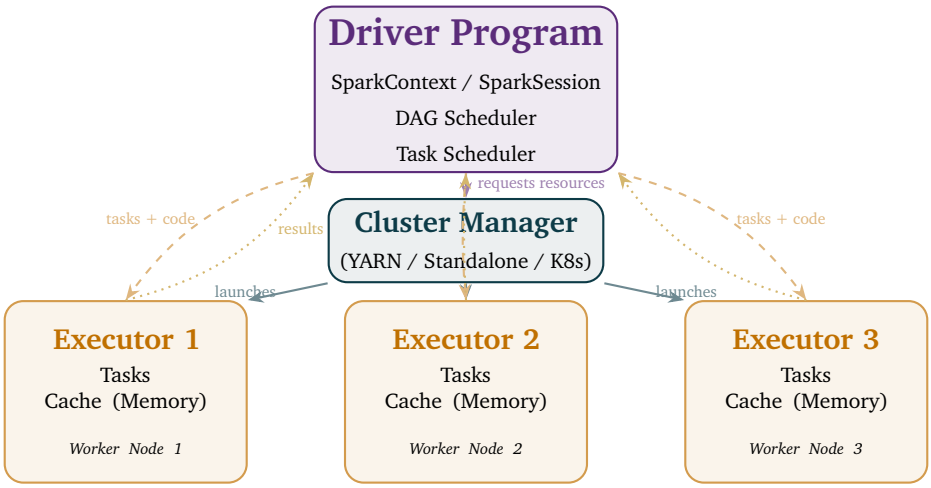


Figure 3.2. *Spark cluster architecture: Driver, Cluster Manager, and Executors*

3.3.2 Lazy Evaluation and DAG Scheduling

Spark does *not* execute transformations immediately. Instead it builds a **Directed Acyclic Graph (DAG)** of the logical plan and only computes when an *action* (e.g., `count()`, `show()`, `save()`) is called.

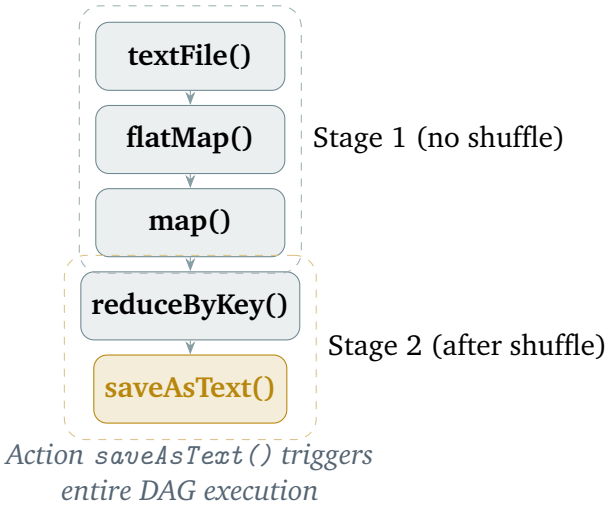


Figure 3.3. *Spark DAG: transformations are lazy; stages split at shuffle boundaries*

Chapter 4

Spark Installation

4.1 Prerequisites

- Java 17 installed (Hadoop prerequisite covers this)
- Python 3.8+ for PySpark
- `JAVA_HOME` set (verify: `echo $JAVA_HOME`)

4.2 Windows Installation (WSL2)

All Spark commands run inside your WSL2 Ubuntu terminal - the same environment you set up for Hadoop. Open Ubuntu from Start Menu before proceeding.

4.3 Linux Installation (Fedora / Ubuntu Native)

All Spark commands run in your native Linux terminal - the same environment you set up for Hadoop.

4.3.1 Method A - Install via pip (Easiest for students)

Listing 4.1. *Install PySpark via pip*

```
# Ensure Python 3 and pip are available
# \lin On Ubuntu (WSL2/Native):
sudo apt install python3 python3-pip -y

# \lin On Fedora:
sudo dnf install python3 python3-pip -y

python3 --version   # Should be 3.8+

# Install PySpark (includes Spark binaries)
pip3 install pyspark

# Verify
python3 -c "import pyspark; print(pyspark.__version__)"
```

4.3.2 Method B - Manual Spark Installation (Recommended for full features)

Listing 4.2. *Download and install Spark 3.5.x manually*

```
cd ~
wget
   https://downloads.apache.org/spark/spark-3.5.1/spark-3.5.1-bin-hadoop3.tg
tar -xzf spark-3.5.1-bin-hadoop3.tgz
mv spark-3.5.1-bin-hadoop3 spark
```

Listing 4.3. *Set Spark environment variables (add to ~/.bashrc)*

```
cat >> ~/.bashrc << 'EOF'

# -- Spark Environment --
export SPARK_HOME=$HOME/spark
export PATH=$PATH:$SPARK_HOME/bin:$SPARK_HOME/sbin
export PYSPARK_PYTHON=python3
EOF

source ~/.bashrc
spark-submit --version    # Verify installation
```

On Fedora, Python 3 is typically pre-installed. If `pip3` is missing, run `sudo dnf install python3-pip -y`. On Ubuntu, use `sudo apt install python3-pip -y`.

4.4 Verify Spark is Working

4.4.1 PySpark Interactive Shell

Listing 4.4. *Launch PySpark shell*

```
pyspark --master local[*]
# Web UI available at http://localhost:4040 while session is
   active
```

Inside the shell:

Listing 4.5. *Quick test in PySpark shell*

```
# SparkContext is automatically available as 'sc'
data = [1, 2, 3, 4, 5]
rdd = sc.parallelize(data)
print(rdd.map(lambda x: x**2).collect())
# Output: [1, 4, 9, 16, 25]
```

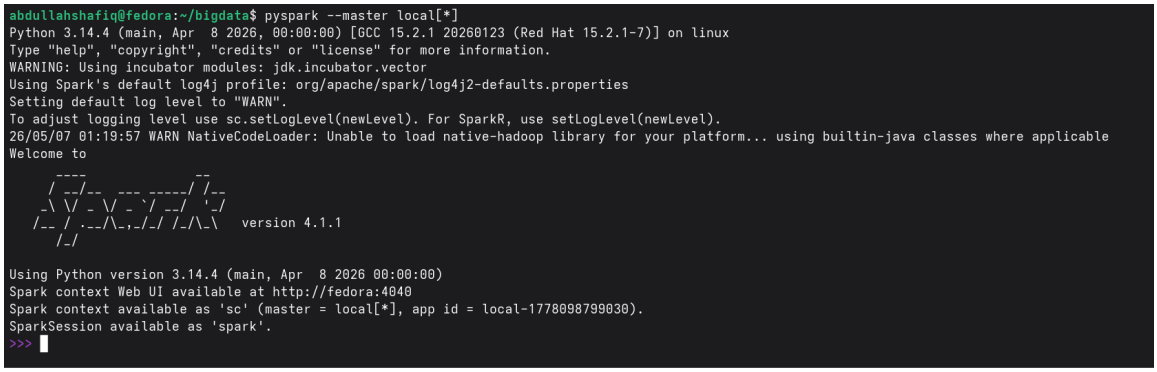


Figure 4.1. *PySpark interactive shell startup showing SparkContext URL at* `localhost:4040`

4.4.2 Submit a Standalone Script

Listing 4.6. *hello_spark.py - test script*

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("HelloSpark") \
    .master("local[*]") \
    .getOrCreate()

sc = spark.sparkContext
sc.setLogLevel("WARN")    # Suppress verbose INFO logs

nums = sc.parallelize(range(1, 101))
total = nums.reduce(lambda a, b: a + b)
print(f"\n>>> Sum of 1 to 100 = {total}\n")    # Should be 5050

spark.stop()
```

Listing 4.7. *Run the test script*

```
spark-submit hello_spark.py
# OR if using pip install method:
python3 hello_spark.py
```

4.5 Common Spark Installation Errors

Error: `JAVA_HOME is not set`

Fix: Ensure `JAVA_HOME` is exported in `~/.bashrc` and you ran `source ~/.bashrc`. Also set it inside `$SPARK_HOME/conf/spark-env.sh`:

```
cp $SPARK_HOME/conf/spark-env.sh.template
   $SPARK_HOME/conf/spark-env.sh
echo "export JAVA_HOME=/usr/lib/jvm/java-17-openjdk" \
  >> $SPARK_HOME/conf/spark-env.sh
```

Error: `Address already in use` **(port 4040)**

Cause: Another Spark session is running.
Fix: Kill the old session or add `.config("spark.ui.port","4041")` to your SparkSession builder.

PySpark: `Python worker failed to connect back`

Cause: `PYSPARK_PYTHON` points to wrong Python binary.
Fix:

```
export PYSPARK_PYTHON=$(which python3)
export PYSPARK_DRIVER_PYTHON=$(which python3)
```

WSL2: Spark UI blank or unreachable

Cause: WSL2 binds to WSL-internal IP, not `localhost`.
Fix: Get WSL IP via `hostname -I` and open `http://<wsl-ip>:4040` in your Windows browser. Alternatively, add to `spark-defaults.conf`:

```
spark.driver.host    localhost
spark.driver.bindAddress  0.0.0.0
```

Spark UI: `localhost:4040` **not loading**

Cause: Firewall blocking the port.
Fix: Open the port temporarily with:

```
sudo firewall-cmd --add-port=4040/tcp --zone=public
```

Chapter 5

PySpark Programming Guide

5.1 RDDs - Resilient Distributed Datasets

5.1.1 What is an RDD?

An RDD is Spark's fundamental distributed data structure:

- **Resilient** - lost partitions are recomputed via lineage
- **Distributed** - data is split into partitions across executors
- **Dataset** - typed collection of elements

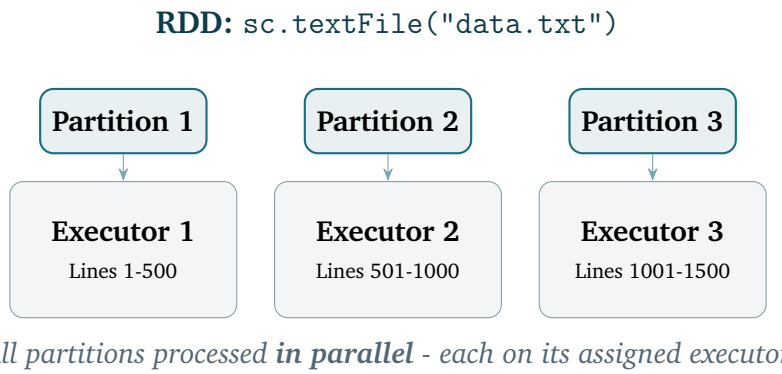


Figure 5.1. *RDD partitioned across three executors*

5.1.2 Creating RDDs

Listing 5.1. *Three ways to create RDDs*

```
from pyspark.sql import SparkSession

spark =
   SparkSession.builder.appName("RDDs").master("local[*]").getOrCreate()
sc = spark.sparkContext

# 1. Parallelize an in-memory collection
data = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
rdd = sc.parallelize(data, numSlices=3)    # 3 partitions

# 2. From a text file (each line = one element)
text_rdd = sc.textFile("data.txt")

# 3. Transform an existing RDD
doubled_rdd = rdd.map(lambda x: x * 2)
```

5.1.3 Key Transformations and Actions

Operation	Type	Description
`map(f)`	Transform	Apply `f` to every element
`filter(f)`	Transform	Keep elements where `f` returns True
`flatMap(f)`	Transform	Map then flatten (great for splitting text)
`reduceByKey(f)`	Transform	Aggregate values per key (pre-aggregates!)
`groupByKey()`	Transform	Group values by key (avoid for large data)
`distinct()`	Transform	Remove duplicate elements
`collect()`	Action	Bring all data to driver (small data only!)
`count()`	Action	Count elements
`take(n)`	Action	Return first *n* elements
`reduce(f)`	Action	Aggregate to single value
`saveAsTextFile(p)`	Action	Write to disk

Table 5.1. *RDD transformations (lazy) vs. actions (trigger execution)*

Listing 5.2. *WordCount with RDDs*

```
text_rdd = sc.textFile("sample.txt")

word_counts = (
    text_rdd
    .flatMap(lambda line: line.lower().split())    # split into
       words
    .map(lambda word: (word, 1))                   # create
       (word,1) pairs
    .reduceByKey(lambda a, b: a + b)               # sum counts
    .sortBy(lambda kv: kv[1], ascending=False)     # sort
       descending
)

word_counts.take(10)    # Show top 10 words
```

5.2 DataFrames and Spark SQL

DataFrames are the preferred API for most tasks. They offer:

- **Schema awareness** - named, typed columns

- **Catalyst optimizer** - automatic query optimization
- **Pandas-like API** with SQL support
- Significantly better performance than raw RDDs for structured data

5.2.1 Creating DataFrames

Listing 5.3. *Multiple ways to create DataFrames*

```
# From a Python list
data = [("Alice", 30, "Engineering"),
        ("Bob",   25, "Marketing"),
        ("Carla", 35, "Engineering")]
columns = ["name", "age", "dept"]
df = spark.createDataFrame(data, columns)
df.show()

# From CSV
df_csv = spark.read.csv("employees.csv", header=True,
   inferSchema=True)

# From JSON
df_json = spark.read.json("employees.json")

# From Parquet (columnar format - best for Spark)
df_pq = spark.read.parquet("employees.parquet")
```

```
>>> data = [("Alice", 30, "Engineering"),("Bob",25,"Marketing"),
...         ("Carla",35, "Engineering")]
>>> df = spark.createDataFrame(data, ["name","age","depth"])
>>> df.show()
+-----+---+-----------+
| name|age|      depth|
+-----+---+-----------+
|Alice| 30|Engineering|
|  Bob| 25|  Marketing|
|Carla| 35|Engineering|
+-----+---+-----------+

>>> df.groupBy("depth").avg("age").show()
+-----------+--------+
|      depth|avg(age)|
+-----------+--------+
|Engineering|    32.5|
|  Marketing|    25.0|
+-----------+--------+

>>>
```

Figure 5.2. *PySpark DataFrame operations: `show()` and `groupBy().avg()` in the interactive shell*

5.2.2 DataFrame Operations

Listing 5.4. *Common DataFrame transformations*

```
from pyspark.sql import functions as F

# Select columns
df.select("name", "age").show()

```

```
# Filter rows
df.filter(df.age > 28).show()
# or using SQL-string syntax:
df.filter("age > 28").show()

# Add / modify columns
df = df.withColumn("age_plus_5", df.age + 5)
df = df.withColumn("dept_upper", F.upper(df.dept))

# GroupBy and aggregate
df.groupBy("dept").agg(
    F.avg("age").alias("avg_age"),
    F.count("name").alias("headcount")
).show()

# Join two DataFrames
dept_df = spark.createDataFrame(
    [("Engineering", "San Francisco"), ("Marketing", "New
       York")],
    ["dept", "city"]
)
joined = df.join(dept_df, on="dept", how="inner")
joined.show()

# Sort
df.orderBy(F.desc("age")).show()
```

5.2.3 Spark SQL

Listing 5.5. *Register a DataFrame and query it with SQL*

```
df.createOrReplaceTempView("employees")

result = spark.sql("""
    SELECT   dept,
             AVG(age)   AS avg_age,
             COUNT(*)   AS total
    FROM     employees
    WHERE    age > 24
    GROUP BY dept
    ORDER BY avg_age DESC
""")
result.show()
```

5.3 Data Sources and File I/O

Listing 5.6. *Reading and writing common formats*

```
# - Reading -
```

```
df = spark.read.csv("data.csv", header=True, inferSchema=True)
df = spark.read.json("data.json")
df = spark.read.parquet("data.parquet")  # Best format for Spark
df = spark.read.orc("data.orc")

# - Writing -
df.write.mode("overwrite").csv("output_csv")
df.write.mode("overwrite").json("output_json")
df.write.mode("overwrite").parquet("output_parquet")

# Partition output by a column (creates sub-folders)
df.write.partitionBy("dept").parquet("output_by_dept")

# Save modes:
#   overwrite  -- replace existing data
#   append     -- add to existing
#   ignore     -- do nothing if exists
#   error      -- raise exception if exists (default)
```

5.4 Performance Optimization

5.4.1 Partitioning Strategy

Listing 5.7. *Managing DataFrame partitions*

```
# Check current partition count
print(df.rdd.getNumPartitions())

# Increase partitions (causes a shuffle - expensive)
df = df.repartition(100)

# Decrease partitions without full shuffle (cheap)
df = df.coalesce(10)

# Rule of thumb: aim for 128 MB per partition
```

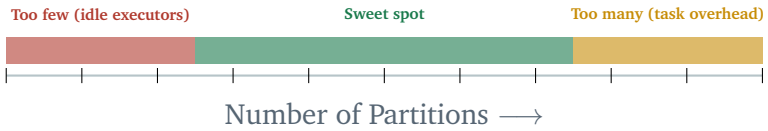


Figure 5.3. *Partition count directly affects cluster efficiency*

5.4.2 Caching

Listing 5.8. *Cache DataFrames used multiple times*

```
df.cache()
# OR with explicit storage level:
```

```
from pyspark import StorageLevel
df.persist(StorageLevel.MEMORY_AND_DISK)

count1 = df.count()     # Triggers computation + caches result
count2 = df.filter("age > 25").count()    # Uses cache -- fast!

df.unpersist()    # Release cache when no longer needed
```

5.4.3 Avoid Common Anti-Patterns

Avoid	Prefer
`df.collect()` on large data	`df.show()`, `df.take(n)`
UDFs (Python overhead)	Built-in `pyspark.sql.functions`
`groupByKey()` then sum	`reduceByKey()` (pre-aggregates)
Multiple redundant sorts	Sort once, reuse cached result
Select all columns, filter late	Select only needed columns *first*

5.5 Performance Optimization

5.5.1 Partitioning Strategy

Listing 5.9. *Managing DataFrame partitions*

```
# Check current partition count
print(df.rdd.getNumPartitions())

# Increase partitions (causes a shuffle - expensive)
df = df.repartition(100)

# Decrease partitions without full shuffle (cheap)
df = df.coalesce(10)

# Rule of thumb: aim for 128 MB per partition
```



Figure 5.4. *Partition count directly impacts cluster efficiency*

5.5.2 Understanding Shuffles

A **shuffle** redistributes data across all partitions - it is the most expensive operation in Spark because it involves disk I/O, serialisation, and network transfer.

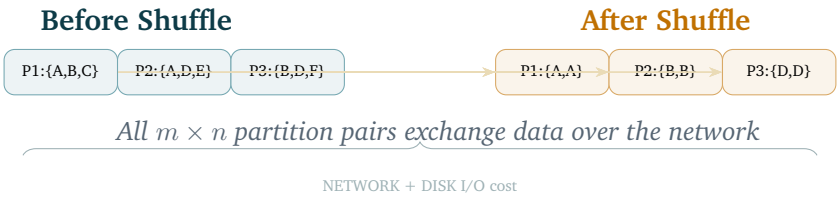


Figure 5.5. *A shuffle is an all-to-all data exchange - minimize it*

Listing 5.10. *Shuffle best practices*

```
# BAD: groupByKey shuffles ALL values, then aggregates
rdd.groupByKey().mapValues(sum)

# GOOD: reduceByKey pre-aggregates within each partition first
rdd.reduceByKey(lambda a, b: a + b)

# Avoid multiple shuffles in a chain
# BAD: two separate shuffles
df.groupBy("dept").count().orderBy("count")

# BETTER: combine with single pass where possible
# Use broadcast join for small tables (avoids shuffle on large
    side)
from pyspark.sql.functions import broadcast
result = large_df.join(broadcast(small_df), "id")
```

5.5.3 Caching

Listing 5.11. *Cache DataFrames that are reused*

```
df.cache()
# OR with explicit storage level:
from pyspark import StorageLevel
df.persist(StorageLevel.MEMORY_AND_DISK)

count1 = df.count()     # Triggers computation + caches result
count2 = df.filter("age > 25").count()    # Uses cache -- fast

df.unpersist()    # Release cache when no longer needed
```

5.5.4 Catalyst Optimizer and Tungsten

Modern Spark includes two automatic optimisation layers that you get for free when using DataFrames and SQL (not raw RDDs):

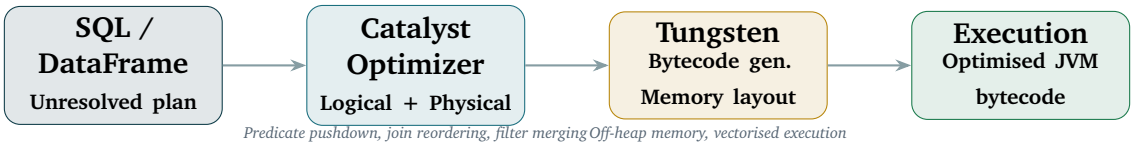


Figure 5.6. *Catalyst + Tungsten: automatic query and execution optimisation*

5.5.5 Avoid Common Anti-Patterns

Anti-Pattern	Better Approach
`df.collect()` on large data	`df.show()` or `df.take(n)`
Python UDFs (serialization overhead)	Built-in `pyspark.sql.functions`
`groupByKey()` then aggregate	`reduceByKey()` (pre-aggregates)
Multiple redundant shuffles	Sort once, reuse cached result
Select all columns, filter late	Select only needed columns first
Large table join (no broadcast)	`broadcast(small_df)` in join
`count()` in a loop	Compute outside the loop, cache result

5.6 Structured Streaming

5.6.1 Batch vs. Streaming

Aspect	Batch	Streaming
Data	Bounded (fixed dataset)	Unbounded (never-ending stream)
Latency	Minutes to hours	Milliseconds to seconds
Trigger	Manual / scheduled	Continuous micro-batches
Examples	Daily ETL, reports	Fraud detection, live dashboards
Spark API	`spark.read.*`	`spark.readStream.*`

Table 5.2. *Batch vs. streaming: key differences*

Spark Structured Streaming treats a live stream as an *unbounded table* that grows with each micro-batch. You write the same DataFrame API as batch code - Spark handles the incremental execution automatically.

5.6.2 Streaming Pipeline Example

Listing 5.12. *Real-time word count from a network socket*

```
from pyspark.sql import SparkSession
from pyspark.sql import functions as F

spark = SparkSession.builder \
    .appName("StreamingWordCount") \
    .master("local[*]") \
    .getOrCreate()
spark.sparkContext.setLogLevel("WARN")

# 1. Read streaming source
stream_df = (
    spark.readStream
    .format("socket")
    .option("host", "localhost")
    .option("port", 9999)
    .load()
```

```
)

# 2. Transform: split into words, count
words = stream_df.select(
    F.explode(F.split(F.col("value"), " ")).alias("word")
)
word_counts = words.groupBy("word").count()

# 3. Write to console (complete = reprint entire table each batch)
query = (
    word_counts.writeStream
    .format("console")
    .outputMode("complete")
    .trigger(processingTime="5 seconds")
    .start()
)
query.awaitTermination()
```

Test this pipeline

Open a second terminal and run: `nc -lk 9999`
Type words and press Enter. Watch counts update in the Spark terminal every 5 s.

5.6.3 Output Modes and Sinks

Output Mode	Writes	Use when
`append`	New rows only	Stateless ops (filter, map)
`complete`	Entire result table	Aggregations (count, sum)
`update`	Changed rows only	Stateful + efficient sinks

Common sinks: `console` (debug), `parquet/delta` (storage), `kafka` (stream-to-stream), `jdbc` (database), `memory` (testing).

5.7 MLlib - Machine Learning with Spark

Listing 5.13. *ML Pipeline: Logistic Regression example*

```
from pyspark.ml import Pipeline
from pyspark.ml.feature import StringIndexer, VectorAssembler, StandardScaler
from pyspark.ml.classification import LogisticRegression
from pyspark.ml.evaluation import BinaryClassificationEvaluator

# Load data
df = spark.read.csv("data.csv", header=True, inferSchema=True)

# Stage 1: Encode categorical column
```

```
indexer = StringIndexer(inputCol="category",
    outputCol="category_idx")

# Stage 2: Assemble all features into a single vector
assembler = VectorAssembler(
    inputCols=["category_idx", "feature1", "feature2"],
    outputCol="raw_features"
)

# Stage 3: Scale features
scaler = StandardScaler(inputCol="raw_features",
    outputCol="features")

# Stage 4: Classifier
lr = LogisticRegression(featuresCol="features",
    labelCol="label", maxIter=20)

# Combine into pipeline
pipeline = Pipeline(stages=[indexer, assembler, scaler, lr])

# Train / test split
train, test = df.randomSplit([0.8, 0.2], seed=42)

# Fit model
model = pipeline.fit(train)

# Evaluate
predictions = model.transform(test)
evaluator = BinaryClassificationEvaluator(labelCol="label")
auc = evaluator.evaluate(predictions)
print(f"AUC-ROC: {auc:.4f}")

# Save and reload model
model.save("my_lr_model")
```

Chapter 6

Spark on Hadoop YARN

Running Spark on YARN lets you share the same cluster between Hadoop and Spark workloads. YARN manages resources; Spark handles execution. This is the most common production setup.

6.1 How Spark Integrates with YARN

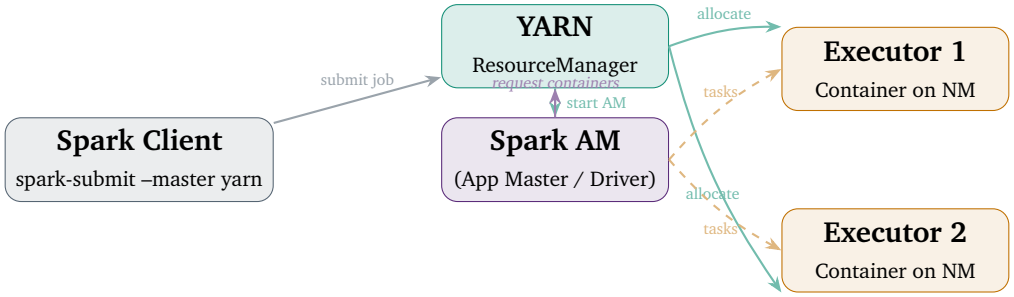


Figure 6.1. *Spark on YARN: the ApplicationMaster acts as the Spark Driver*

Spark supports two YARN deployment modes:

Mode	Driver runs on	Best for
`-deploy-mode client`	Your submitting machine	Interactive/debugging
`-deploy-mode cluster`	A YARN NodeManager	Production batch jobs

6.2 Submitting Spark Jobs to YARN

Hadoop must be running (`start-dfs.sh && start-yarn.sh`) before submitting Spark jobs to YARN.

Listing 6.1. *Submit a PySpark script to YARN (client mode)*

```
# Basic YARN submission
spark-submit \
  --master yarn \
  --deploy-mode client \
  --num-executors 2 \
  --executor-memory 1g \
  --executor-cores 1 \
  your_script.py
```

```
# Cluster mode (driver on cluster node)
spark-submit \
  --master yarn \
  --deploy-mode cluster \
  --num-executors 3 \
  --executor-memory 2g \
  --driver-memory 1g \
  your_script.py
```

Listing 6.2. *Configure YARN endpoint inside Spark*

```
# Option 1: Set in your script's SparkSession
spark = SparkSession.builder \
    .appName("MyYARNJob") \
    .master("yarn") \
    .config("spark.executor.memory", "1g") \
    .config("spark.executor.instances", "2") \
    .getOrCreate()

# Option 2: Add to $SPARK_HOME/conf/spark-defaults.conf
# spark.master            yarn
# spark.executor.memory 1g
```

Listing 6.3. *Useful YARN + Spark monitoring commands*

```
# List YARN applications (running and completed)
yarn application -list

# Get application logs
yarn logs -applicationId application_XXXXXXX_XXXX

# Kill a stuck application
yarn application -kill application_XXXXXXX_XXXX

# Check cluster resource availability
yarn node -list
```

6.3 Tuning Spark Resource Allocation

Choosing the right executor configuration avoids two common traps: **under-utilisation** (too few executors) and **memory overhead** (too large executors).

Parameter	Typical default	Guidance
`-executor-memory`	1g	Start with half available node RAM
`-executor-cores`	1	2–5 cores/executor; more causes GC pressure
`-num-executors`	dynamic	# nodes × (cores/node ÷ executor-cores)
`-driver-memory`	1g	Increase if collecting large results
`spark.default.parallelism`	2× cores	Set to 2–4× total executor cores

Table 6.1. *Key Spark resource configuration parameters*

Enable **dynamic allocation** to let YARN scale executors automatically:

```
spark.dynamicAllocation.enabled          true
spark.dynamicAllocation.minExecutors     1
spark.dynamicAllocation.maxExecutors     10
spark.shuffle.service.enabled            true
```

6.4 Spark Web UI on YARN

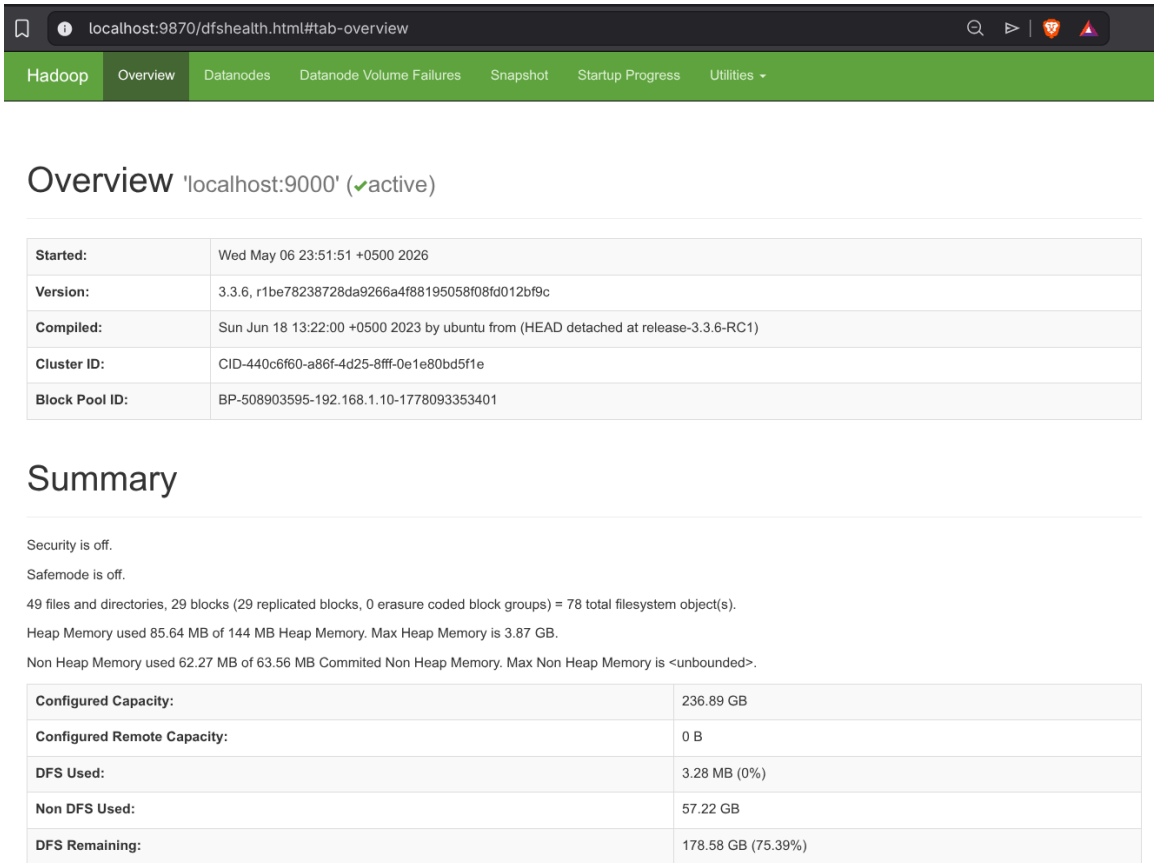


Figure 6.2. *Hadoop NameNode Web UI at `localhost:9870` showing cluster summary and storage stats*

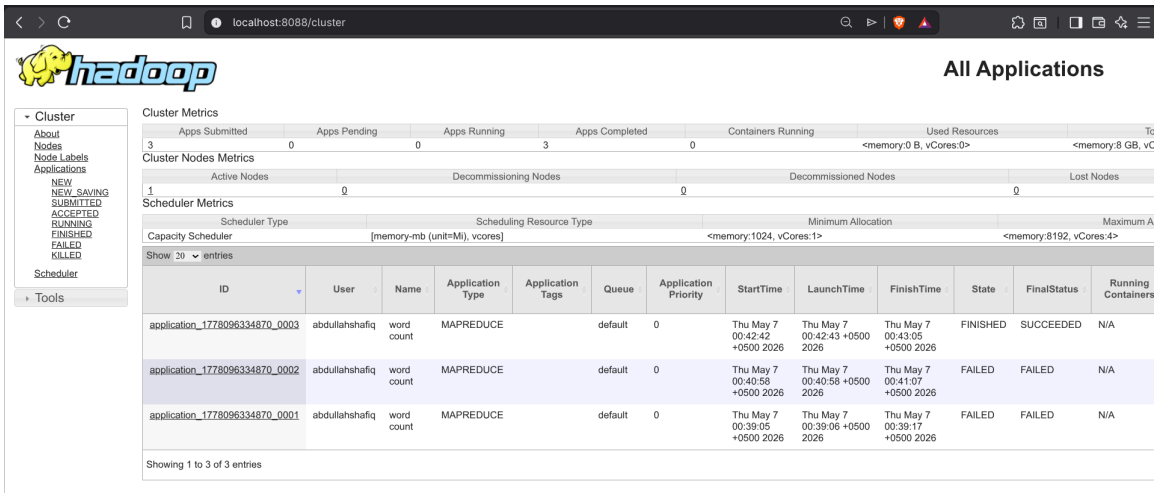


Figure 6.3. *YARN ResourceManager Web UI at `localhost:8088` showing completed application*

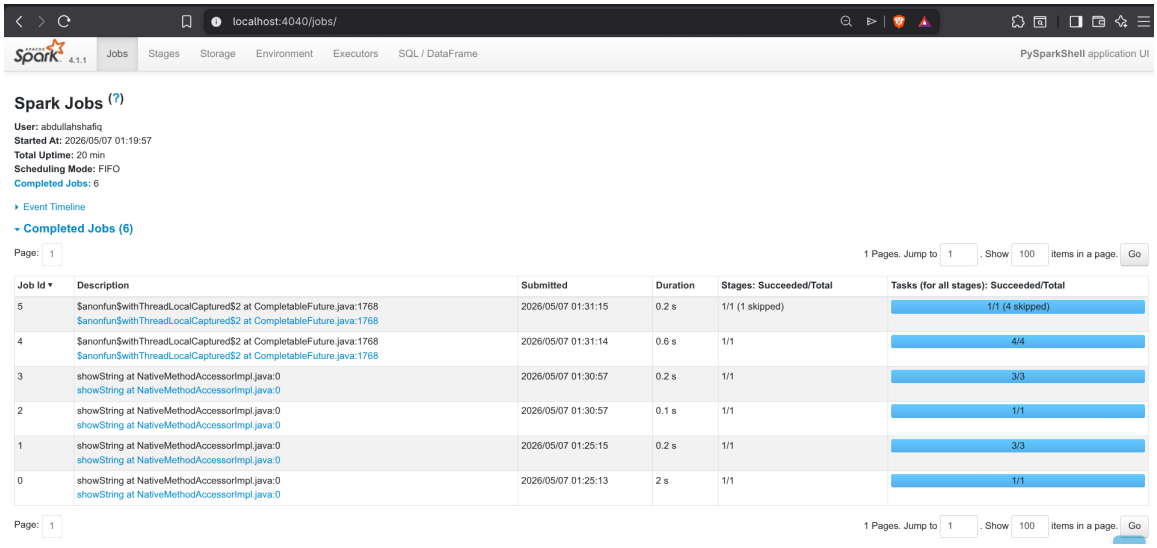


Figure 6.4. *Spark Web UI at `localhost:4040` showing completed jobs, stages, and executor memory*

Interface	URL
YARN ResourceManager	`http://localhost:8088`
Spark Application UI	`http://localhost:4040` (client mode)
Spark History Server	`http://localhost:18080` (after job ends)
HDFS NameNode	`http://localhost:9870`

Listing 6.4. *Start the Spark History Server (keeps UI after job ends)*

```
# Create log directory on HDFS
hdfs dfs -mkdir -p /spark-logs
hdfs dfs -chmod 777 /spark-logs

# In $SPARK_HOME/conf/spark-defaults.conf:
# spark.eventLog.enabled           true
# spark.eventLog.dir
    hdfs://localhost:9000/spark-logs
# spark.history.fs.logDirectory
    hdfs://localhost:9000/spark-logs

# Start history server
$SPARK_HOME/sbin/start-history-server.sh
```

Chapter 7

Practice Projects

7.1 Project 1 - ETL Pipeline with Spark

Build a complete **Extract, Transform, Load** pipeline:

Listing 7.1. *End-to-end ETL pipeline*

```
from pyspark.sql import SparkSession
from pyspark.sql import functions as F

spark =
   SparkSession.builder.appName("ETL").master("local[*]").getOrCreate()
sc = spark.sparkContext
sc.setLogLevel("WARN")

# -- 1. EXTRACT --
raw = spark.read.csv("sales.csv", header=True, inferSchema=True)
print(f"Raw rows: {raw.count()}")
raw.printSchema()

# -- 2. TRANSFORM --
clean = (
    raw
    .dropna(subset=["order_id", "amount"])     # Remove rows
       with null keys
    .withColumn("date", F.to_date("date_str", "yyyy-MM-dd"))
    .withColumn("amount", F.col("amount").cast("double"))
    .withColumn("revenue_tier",
        F.when(F.col("amount") > 10000, "High")
         .when(F.col("amount") > 5000,  "Medium")
         .otherwise("Low")
    )
    .filter(F.col("date") >= "2024-01-01")     # Last-year data
       only
)

# Aggregate by department and month
summary = (
    clean
    .withColumn("month", F.month("date"))
    .groupBy("department", "month")
    .agg(
        F.sum("amount").alias("total_revenue"),
        F.count("order_id").alias("num_orders"),
```

```
        F.avg("amount").alias("avg_order_value")
    )
    .orderBy("department", "month")
)

# -- 3. LOAD --
summary.write.mode("overwrite").parquet("output/sales_summary")
print("ETL complete.")
spark.stop()
```

7.2 Project 2 - WordCount: RDD vs. DataFrame

Listing 7.2. *Compare RDD and DataFrame approaches*

```
import time

# -- RDD approach --
t0 = time.time()
rdd_counts = (
    sc.textFile("book.txt")
    .flatMap(lambda l: l.lower().split())
    .map(lambda w: (w, 1))
    .reduceByKey(lambda a, b: a + b)
    .sortBy(lambda kv: kv[1], ascending=False)
)
rdd_counts.saveAsTextFile("output/wc_rdd")
t1 = time.time()
print(f"RDD time: {t1 - t0:.2f}s")

# -- DataFrame approach --
t2 = time.time()
(
    spark.read.text("book.txt")
    .select(F.explode(F.split(F.lower(F.col("value")),
      r"\W+")).alias("word"))
    .filter(F.col("word") != "")
    .groupBy("word").count()
    .orderBy(F.desc("count"))
    .write.mode("overwrite").parquet("output/wc_df")
)
t3 = time.time()
print(f"DataFrame time: {t3 - t2:.2f}s")
# DataFrame is typically 5-10x faster due to Catalyst
```

7.3 Project 3 - Analysing a Real Dataset

This project uses the publicly available NYC Taxi dataset to demonstrate end-to-end Spark skills: loading, cleaning, aggregating, and visualising.

Listing 7.3. *NYC Taxi trip analysis with PySpark*

```
from pyspark.sql import SparkSession
from pyspark.sql import functions as F
from pyspark.sql.types import DoubleType

spark = SparkSession.builder \
    .appName("NYC Taxi Analysis") \
    .master("local[*]") \
    .getOrCreate()
spark.sparkContext.setLogLevel("WARN")

# 1. Load data (download sample CSV from NYC TLC website)
raw = spark.read.csv("yellow_tripdata_2024-01.csv",
                     header=True, inferSchema=True)
print(f"Total trips: {raw.count():,}")
raw.printSchema()

# 2. Clean: drop nulls, filter outliers
clean = (
    raw
    .dropna(subset=["tpep_pickup_datetime",
                    "trip_distance", "total_amount"])
    .filter(F.col("trip_distance") > 0)
    .filter(F.col("total_amount") > 0)
    .filter(F.col("total_amount") < 500)    # Remove anomalies
)
print(f"Clean trips: {clean.count():,}")

# 3. Feature engineering
trips = clean.withColumn(
    "hour_of_day", F.hour("tpep_pickup_datetime")
).withColumn(
    "day_of_week", F.dayofweek("tpep_pickup_datetime")
).withColumn(
    "speed_mph",
    (F.col("trip_distance") /
     (F.unix_timestamp("tpep_dropoff_datetime") -
      F.unix_timestamp("tpep_pickup_datetime")) *
         3600).cast(DoubleType())
)

# 4. Aggregate: trips by hour of day
hourly = (
    trips
    .groupBy("hour_of_day")
    .agg(
        F.count("*").alias("num_trips"),
        F.avg("trip_distance").alias("avg_dist_miles"),
        F.avg("total_amount").alias("avg_fare_usd"),
        F.avg("speed_mph").alias("avg_speed_mph")
    )
```

```
    .orderBy("hour_of_day")
)
hourly.show(24)

# 5. Find busiest pickup zones
if "PULocationID" in trips.columns:
    top_zones = (
        trips.groupBy("PULocationID")
        .count()
        .orderBy(F.desc("count"))
        .limit(10)
    )
    top_zones.show()

# 6. Save results
hourly.write.mode("overwrite").parquet("output/taxi_hourly_stats")
print("Analysis complete!")
spark.stop()
```

Download a sample file:
`wget https://d37ci6vzurychx.cloudfront.net/trip-data/yellow_tripdata_2024-01.parquet`
Use `spark.read.parquet()` for the Parquet version - it is faster and typed.

7.4 Managed Spark and Cloud Platforms

Once you are comfortable with local Spark, real workloads move to managed platforms:

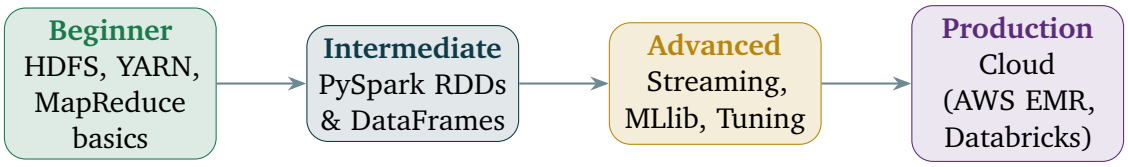
Platform	Provider	Key features
Databricks	Independent / Azure	Collaborative notebooks, Delta Lake, auto-scaling, MLflow integration
AWS EMR	Amazon	Managed Hadoop+Spark on EC2; integrates with S3, Glue
Google Dataproc	Google	Fast cluster startup; integrates BigQuery and GCS
Azure Synapse	Microsoft	Spark + SQL pools; Power BI integration
Apache Spark on K8s	Self-hosted	Containerised; fine-grained multi-tenancy

Table 7.1. *Managed Spark platforms for production workloads*

Topic	Hadoop	Spark
Storage	HDFS	HDFS, S3, GCS, local
Processing	MapReduce (disk)	In-memory RDD/DF
Speed	Moderate	10–100× faster
Streaming	No	Structured Streaming
ML	Mahout (limited)	MLlib (rich)
SQL	Hive (separate)	Spark SQL (built-in)
Web UI	:9870, :8088	:4040
Start cmd	`start-dfs.sh`	`spark-submit`
Stop cmd	`stop-dfs.sh`	`query.stop()`

Table 7.2. *Hadoop vs. Spark at a glance*

7.5 Recommended Learning Path



Appendix A: Quick Cheat Sheet

A.1 Hadoop Commands

Command	Description
`start-dfs.sh`	Start HDFS daemons (NameNode + DataNode)
`start-yarn.sh`	Start YARN daemons (RM + NM)
`stop-dfs.sh / stop-yarn.sh`	Stop respective daemons
`jps`	List all running Java processes (verify daemons)
`hdfs namenode -format`	Format NameNode (first time only!)
`hdfs dfsadmin -report`	Cluster capacity and live DataNodes
`hdfs dfsadmin -safemode leave`	Manually exit safe mode
`hdfs dfs -ls -R /`	Recursive directory listing
`hdfs dfs -du -h /path`	Disk usage in human-readable form
`hdfs dfs -setrep -w 2 /file`	Change replication factor to 2
`hdfs dfs -checksum /file`	Print block checksums
`hadoop jar $JAR class /in /out`	Submit a MapReduce JAR job
`mapred job -list`	List running MapReduce jobs
`mapred job -kill <jobId>`	Kill a stuck MapReduce job

A.2 PySpark / Spark-Submit Commands

Command / Snippet	Description
`pyspark -master local[*]`	Launch interactive PySpark shell
`spark-shell`	Scala Spark REPL
`spark-submit script.py`	Submit Python script (local)
`spark-submit -master yarn script.py`	Submit to YARN cluster
`df.printSchema()`	Show DataFrame column types
`df.show(n, truncate=False)`	Display first n rows in full
`df.describe().show()`	Summary statistics
`df.explain()`	Print execution plan (DAG)
`df.rdd.getNumPartitions()`	How many partitions
`df.cache() / df.unpersist()`	Cache or release from memory
`sc.setLogLevel("WARN")`	Suppress verbose INFO logs
`spark.stop()`	Gracefully shut down SparkSession

A.3 Common Error Quick-Fix Index

Error	Fix
`ssh localhost` asks password	`chmod 700 ~/.ssh && chmod 600 ~/.ssh/authorized_keys`
DataNode missing from `jps`	Delete datanode dir, reformat NameNode
NameNode in safe mode	`hdfs dfsadmin -safemode leave`
`JAVA_HOME is not set`	Export `JAVA_HOME` in `~/.bashrc` and `spark-env.sh`
Port 4040 already in use	`.config("spark.ui.port","4041")`
Python worker failed to connect	Set `PYSPARK_PYTHON=$(which python3)`
`OutOfMemoryError` in Driver	Add `-driver-memory 2g` to spark-submit
WSL2: localhost not reachable	Use `hostname -I` IP in Windows browser
Firewall blocking Web UI	`sudo firewall-cmd -add-port=9870/tcp -zone=public`
`Permission denied` on HDFS	`hdfs dfs -chmod 777 /path`
`Address already in use: 9000`	Another Hadoop instance running; `stop-dfs.sh`

A.4 File Format Decision Guide

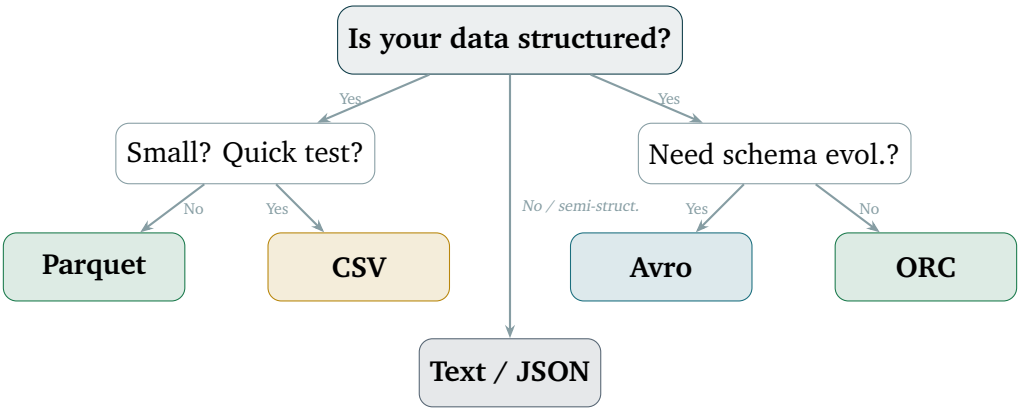


Figure 7.1. *How to choose the right storage format in Spark/Hadoop*

Format	Compress	Splittable	Schema	Best for
CSV	No	Yes	No	Quick exploration, interchange
JSON	No	No	No	REST APIs, semi-structured data
Parquet	Yes	Yes	Yes	Analytical queries on columns
ORC	Yes	Yes	Yes	Hive workloads
Avro	Yes	Yes	Yes	Schema evolution, Kafka

Table 7.3. *Storage format comparison*

References

[1] Apache Software Foundation. (2024). *Apache Hadoop 3.3.x Documentation*. https://hadoop.apache.org/docs/r3.3.6/

[2] Apache Software Foundation. (2025). *Apache Spark 3.5 Documentation*. https://spark.apache.org/docs/latest/

[3] Zaharia, M., *et al.* (2012). Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing. *USENIX NSDI*.

[4] Armbrust, M., *et al.* (2015). Spark SQL: Relational Data Processing in Spark. *ACM SIGMOD*.

[5] Karau, H., Konwinski, A., Wendell, P., & Zaharia, M. (2015). *Learning Spark*. O'Reilly Media.

[6] Fedora Project. (2025). *Fedora Documentation*. https://docs.fedoraproject.org/

[7] Databricks. (2024). *Apache Spark Training - Free Courses*. https://academy.databricks.com/

[8] SparkByExamples. (2025). *PySpark Tutorial with Examples*. https://sparkbyexamples.com/